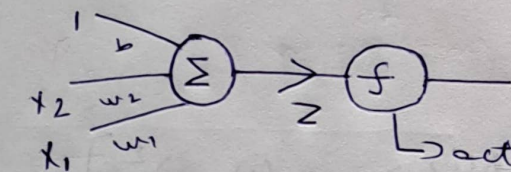


Deep Learning

Types of Neural networks

- ① ANN ② CNN ③ RNN ⑤ LSTM ⑥ GAN ⑦ autoencoders

★ Perceptron: → An algorithm used for supervised machine learning. Its design is in a way that it automatically became the building block of deep learning. Can be called a mathematical model/function.



$$Z = w_1 x_1 + w_2 x_2 + b$$

activation fn

used to bring Z into a given range

Perceptron is a binary classifier.

Perceptron can classify only linear / Sort of linear data

In 2D perceptron is a line

In 3D, a plane

4D onwards, a hyperplane

ways to update the weights —

- ① Perceptron trick: → we start with random values of A, B, C . Then we run a loop & in each loop iteration, we choose a random data point & check if it's in its correct place. If yes, we don't change the A, B, C (weight) values & move the loop further & select a different data point. & if the data point is wrongly classified, we will update the weights in such a way that the classifier line moves closer to the data point & the point becomes correctly classified.

number of epochs for Perceptron trick — either 1000 or until convergence.

number of misclassified points
= 0.

How to identify positive / negative regions / How to label regions →
fit the (x_1, y_1) into the equation $Ax_1 + By_1 + C = 0$ → on the line.
 > 0 → +ve region
 < 0 → -ve region

Transformations →

$\downarrow C$ → line goes up
 $\uparrow C$ → line goes down

① If the change is in C → the line moves parallelly up, down.

② If the change is in B → _____ around x axis.

③ _____ A → _____ y axis.

How to transform? → ① if a -ve point is in the +ve region, we introduce a 1 in its coordinates eg. → $(4, 5) \rightarrow (4, 5, 1)$ & subtract them from the coefficients of the line $(2x + 3y + 5 = 0)$.

$$\begin{array}{r}
 2 \quad 3 \quad 5 \\
 4 \quad 5 \quad 1 \\
 \hline
 -2x - 2y + 4 = 0
 \end{array}$$

② If a +ve point is in the -ve region, we introduce a 1 similarly & we add them to the coefficients of the line.

$$\begin{array}{r}
 2 \quad 3 \quad 5 \\
 + 4 \quad 5 \quad 1 \\
 \hline
 6x + 8y + 6 = 0
 \end{array}$$

new coeff = old coeff - η coordinates

learning rate. → to avoid drastic transformations

Algorithm $\rightarrow$

$$Ax + By + C = 0$$

$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

$$w_0 = C; \quad w_1 = A; \quad w_2 = B$$

here, $w_0 = \text{bias}$

Assuming $x_0 = 1$ for all rows then, $w_0 x_0 + w_1 x_1 + w_2 x_2 = 0$

$$\therefore \sum_{i=0}^m w_i x_i = 0$$

$m = \text{number of features.}$

$\rightarrow$ general eqn of line

$$\begin{bmatrix} w_0 & w_1 & w_2 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = w_0 x_0 + w_1 x_1 + w_2 x_2$$

$$\text{epoch} = 1000; \quad \eta = 0.01$$

for i in range (epochs):

randomly select a data point

if $x_i \in \text{Negative region} \& \sum_{i=0}^m w_i x_i \geq 0 \{$

$$w_{\text{new}} = w_{\text{old}} - \eta x_i \}$$

if $x_i \in \text{Positive region} \& \sum_{i=0}^m w_i x_i < 0 \{$

$$w_{\text{new}} = w_{\text{old}} + \eta x_i \}$$

Simplified algorithm $\rightarrow$ for i in range (epochs)

Randomly select

$$w_{new} = w_{old} + \eta (y_i - \hat{y}_i) x_i$$

$\rightarrow$ actual $\rightarrow$ ~~pred~~ predicted

y_i	$\hat{y}_i$	$y_i - \hat{y}_i$	update
1	1	0	$w_n = w_0$
0	0	0	$w_n = w_0$
1	0	1	$w_n = w_0 + \eta x_i$
0	1	-1	$w_n = w_0 - \eta x_i$

Loss Functions in Perceptron

Problems with perceptron trick $\rightarrow$ ① might / might not give you the optimal values of the weights & bias.

② The model might not converge

Solution $\rightarrow$ loss function $\rightarrow$ way to evaluate the model's performance
 $\hookrightarrow$ minimizing the cost function

Perceptron loss function $\rightarrow$ ① number of misclassified points.

② Distance between the line & the misclassified points.

Actually used loss fn for a perceptron

$$L(\vec{w}, b) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \alpha R(\vec{w}) \quad \rightarrow \text{regularization}$$

$$L(y_i, f(x_i)) = \max(0, -y_i f(x_i))$$

$$\hookrightarrow f(x_i) = \vec{w} \cdot \vec{x} + b$$

$$L = \frac{1}{n} \sum_{i=1}^n \max(0, -y_i f(x_i))$$

n = number of rows in data.

Gradient Descent

$$L = \underset{\vec{w}, b}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \max(0, -y_i f(x_i))$$

$$\text{where } f(x_i) = \vec{w} \cdot \vec{x} + b$$

Start with random values of $\vec{w}, b$

for i in epochs:

$$w_i = w_i + \eta \left[\frac{\partial L}{\partial w_i} \right] \quad // \text{ from the differentiation}$$

$$b = b + \eta \frac{\partial L}{\partial b} \quad \text{until convergence}$$

$$\text{After calculation } \rightarrow \frac{\partial L}{\partial w_i} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i x_i & \text{if } y_i f(x_i) < 0 \end{cases}$$

$$\text{general } \rightarrow \frac{\partial L}{\partial w_j} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i x_{ij} & \text{if } y_i f(x_i) < 0 \end{cases}$$

$$\frac{\partial L}{\partial b} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i & \text{if } y_i f(x_i) < 0 \end{cases}$$

More loss functions $\rightarrow$ ① binary cross entropy used with sigmoid activation function.

② Softmax & Categorical cross entropy.

③ linear & MSE

MultiLayer Perceptron

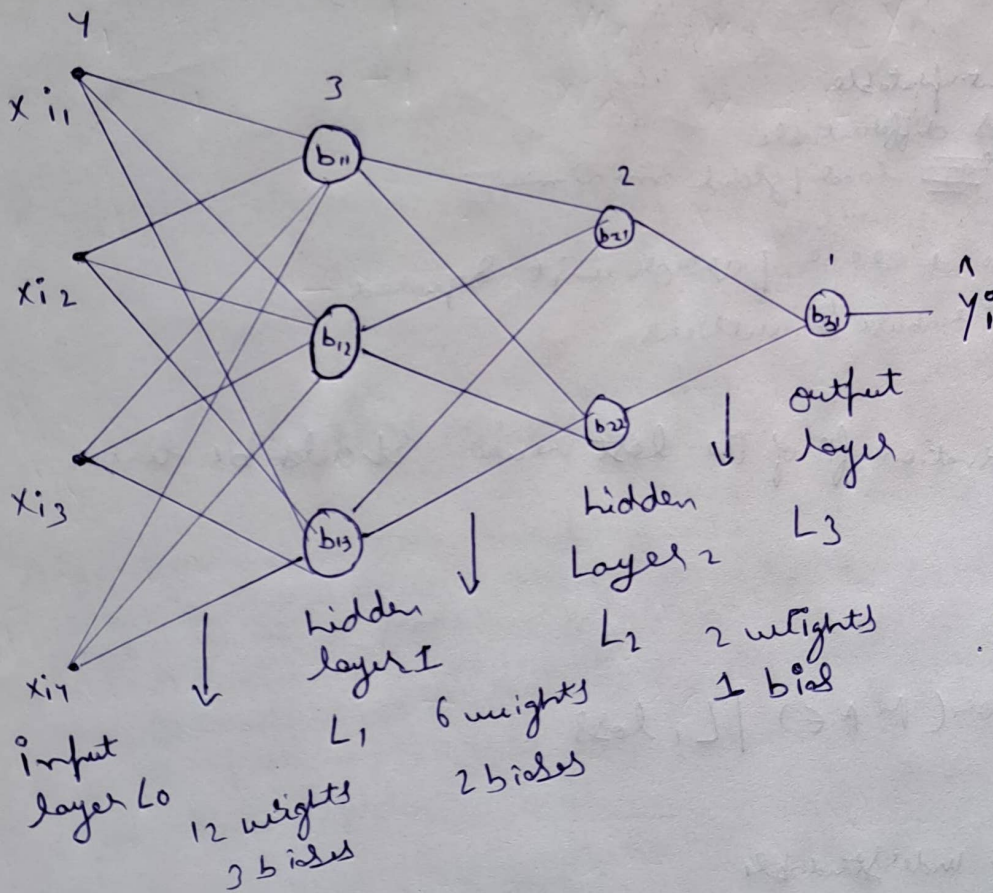
MLP is also called a neural network

Notation $\rightarrow$ Data = $\{ m \times n \}$

$m \rightarrow$ number of rows

$n = 4$ number of features / columns - output label

each row denoted by x_i



$\therefore$ 26 trainable parameters

$b_{ij} =$ bias of i^{th} ~~row~~ ^(layer) & j^{th} node

output = $O_{ij} =$ output of i^{th} ~~row~~ ^(layer) & j^{th} node

weights $\rightarrow w_{ij}^k$

$k =$ the layer in which it's entering

$i =$ current layer's node from which it's originating

$j =$ the node in the layer it's entering

Forward Propagation

Loss functions $\rightarrow$ a method of evaluating how well your algorithm is modelling your dataset. A mathematical function

Types of loss functions in Deep learning $\rightarrow$

① MSE (Mean Squared error) / L_2 loss

$$(y_i - \hat{y}_i)^2$$

Advantages $\rightarrow$

- ① easily interpretable
- ② always differentiable
- ③ one ~~loss~~ local/global minima.

Disadvantages $\rightarrow$

- ① Squared error / error unit Squared
- ② not robust to outliers.

To apply this, the activation f' of the last neuron should be linear activation function.

② Mean absolute error (MAE) / L_1 loss

$$|y_i - \hat{y}_i|$$

Advantages $\rightarrow$

- ① easily understandable
- ② same unit as y_i
- ③ robust to outliers.

Disadvantages $\rightarrow$

- ① not differentiable

③ Huber loss $\rightarrow$

$$\begin{cases} \frac{1}{2} (y - \hat{y})^2 & \text{for } |y - \hat{y}| \leq \delta \\ \delta |y - \hat{y}| - \frac{1}{2} \delta^2 & \text{otherwise.} \end{cases}$$

↑
hyper parameter δ

↓
work well when your data has many outliers.

④ Binary cross entropy $\rightarrow$ also called log loss

$$-y \log(\hat{y}) - (1-y) \log(1-\hat{y})$$

Activation has to be sigmoid in the output layer

Advantage $\rightarrow$ ① easily differentiable

⑤ Disadvantage $\rightarrow$ ① multiple local minimas possible

⑤ Categorical cross entropy $\rightarrow$ used with softmax activation function

$$L = - \sum_{j=1}^K y_j \log(\hat{y}_j) ; K = \text{number of classes in the data}$$

⑥ Sparse categorical cross entropy $\rightarrow$ instead of one hot encoding the labels, we integer encode.

Back Propagation

Algorithm used to train neural networks

Step 0 $\rightarrow$ initialize weights & biases $w = 1, b = 0$

Steps ① we select a row

② Prediction (forward Propagation)

③ Choose a loss function

④ update weights & biases using gradient descent

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w_{\text{old}}}$$

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial L}{\partial b_{\text{old}}}$$

Gradient Descent

An optimization algorithm $\rightarrow$ minimizes the cost function

Types $\rightarrow$ ① batch ② Stochastic ③ Mini batch.

number of epochs $\downarrow$
= number of updates we use the entire dataset

Shuffle the data & calculate $\hat{y}$ for a randomly chosen point & for it only we calculate the loss.
for i in range(epochs) $\rightarrow$ Shuffle

for i in range(1, shape[0])

at the end of the inner loop calculate average loss.

number of updates = epochs $\times$ number of rows

Diff b/w both $\rightarrow$ ① batch gd is faster. SGD is slower

② batch GD $\rightarrow$ we do the algorithm on the entire dataset at once.

epoch = 10

for i in range(10):

$$\hat{y} = w \cdot \text{dot}(x, W) + b$$

n rows

loss is the cumulative loss.

n = number of rows

③ If batch_size = n $\rightarrow$ batch GD
batch_size = 1 $\rightarrow$ SGD

④ SGD converges faster (converges more in same no. of epochs as BGD)

So, SGD $\rightarrow$ converges faster
BGD $\rightarrow$ completely epochs faster

⑤ unstable loss decrease in SGD ; Stable loss decrease in BGD
(rough) (Smooth)

⑥ BGD can be vectorized

Mini Batch Gradient Descent $\rightarrow$ mix of both.

for in range(epochs):
for j in num_of_batches:

1 batch

$\hookrightarrow$ x -feed (vectorize)

$\hookrightarrow$ loss

$\hookrightarrow$ update.

8) if batch-size doesn't divide all the rows perfectly!
= # of rows = 400

batch-size = 150

of batches = $\frac{400}{150} = 2.66$

$\frac{150}{150}$, $\frac{150}{150}$, remaining 100 $\rightarrow$ 3rd batch
 $\downarrow$ $\downarrow$
1st batch 2nd batch

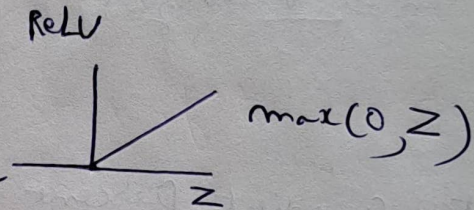
Vanishing Gradient Problem

- ① occurs in gradient based algorithms
- ② occurs in deep neural networks only.
- ③ occurs in mostly (sigmoid) / (tanh) activation functions.

Gradients become so small that they stop updating the weights. or update them very slowly.

Solution → ① Reduce model complexity

② Using ReLU in the hidden layers.



dying ReLU problem
if derivative less than 0 then ReLU makes it 0
& the weights don't get updated.

←
Solution → Leaky ReLU

③ Proper weight initialization → more later

④ batch normalization → more later

⑤ Residual network → more later (in CNN)

Exploding Gradient Problems → opposite to VGP.

Common in RNNs

Improving a Neural Network's Performance

① Fine tuning the hyperparameters

- No. of hidden layers
- no. of layers/neuron.
- learning rate
- optimizer
- batch size
- activation function
- epochs

② By Solving → ① Vanishing / exploding gradient problem.

- ② not enough data
- ③ Slow training
- ④ overfitting

Tuning the hyperparameters

- ① number of hidden layers → use more layers with more neurons per layer. Helps in transfer learning as well.
- ② number of neurons/layer in the hidden layers → subjective.
Should be more than what's needed.
- ③ learning rate / optimizer / slow training / activation functions later
- ④ batch size = use variable learning rate / increasing up the learning rate → low initially, gradual increase with increase in epochs
- ⑤ epochs → use early stopping

Early Stopping

Stop the model training the first overfitting signs appear

callback = EarlyStopping(

monitor = 'val_loss',

accepted

min_delta =

min. improvement

patience =

of epochs with no improvement after which training stops.

verbose =

display message or not

mode = "auto" / "min" / "max"

monitors the quantity, when to stop, auto, on increase or decrease.

baseline → Training stops if the model doesn't show improvement over the baseline usually None.

restore_best_weights = True / False

use callback in model.fit()

Feature Scaling / Normalization

Standardize

$$\frac{X_i - \mu}{\sigma}$$

Normalize

$$\frac{X_i - X_{\min}}{X_{\max} - X_{\min}}$$

Dropouts

we randomly deactivate some neurons / epoch during training only we can't assign the whole 'w' weight to a node as it was not available for the whole time.

$$\therefore w = w(1 - p)$$

$$= w(1 - 0.25)$$

$$= 0.75w$$

p = dropout ratio eg = 0.25

↓
drop 25% neurons / epoch

→ as the neuron was present 75% of the time, $w = 0.75w$

Regularization

Use whenever your model overfits

Types $\rightarrow L_1 = C = \sum L + \frac{1}{2n} \sum_{i=1}^K \|W_i\| \rightarrow L_1 \text{ norm}$

$L_2 = C = \sum L + \frac{1}{2n} \sum_{i=1}^K \|W_i\|^2 \rightarrow L_2 \text{ norm}$

L_2 is more widely used.

Activation Functions

each neuron forms a weighted sum of its inputs & passes the resulting scalar value through a function called activation / transfer function

$$a = g(w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b)$$

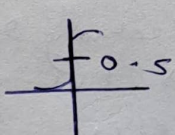
without activation functions, every a n n is a linear regression model only.

Types $\rightarrow$

mean ≈ 0
normalized output

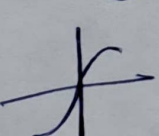
Qualities of an ideal activation function $\rightarrow$ ① Non-linear ② 0-centered
② Differentiable ③ Computationally inexpensive

⑤ non saturating saturating $\rightarrow$ doesn't squeeze the input within a range.

① Sigmoid = $a = \frac{1}{1 + e^{-z}}$ 

Advantages $\rightarrow$ ① output $\rightarrow [0, 1]$ ③ differentiable
② non-linear

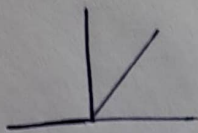
Disadvantages $\rightarrow$ ① Saturating $\rightarrow$ Vanishing gradient problem
② non-0 centered $\rightarrow$ slower training
③ Computationally expensive

② Tanh =  $[-1, 1]$ $a = \frac{(e^z - e^{-z})}{(e^z + e^{-z})}$

Advantages $\rightarrow$ ① non linear, ② differentiable, ③ 0-centered.

Disadvantages $\rightarrow$ ① saturating ② Computationally expensive

③ ReLU



$$a = \max(0, z)$$

Advantages $\rightarrow$ ① non linear ② not saturated in the positive region.
 ③ computationally inexpensive
 ④ fast convergence

Disadvantages $\rightarrow$ ① non differentiable in $[-\infty \text{ to } 0]$

② not 0-centered.

$\swarrow$ solution
 batch-normalization

$\downarrow$ solution

assume $\frac{d}{dz}$ for $< 0 = 0$
 for $\geq 0 = 1$

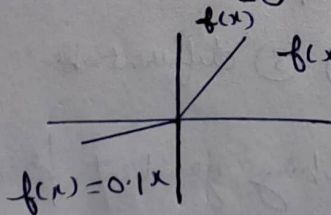
Dying ReLU Problem $\rightarrow$ occurs when a large number of neurons in a neural network using ReLU ~~stop~~ output 0 & stop learning.
 happens when a neuron's input is always -ve as $\text{ReLU} = \max(0, z)$ for -ves it is 0.

how can z be -ve? $\rightarrow$ ① when learning rate is very high
 $z = w_1 x_1 + w_2 x_2 + b < 0$ ② high -ve bias

Solutions $\rightarrow$ ① set small learning rates.
 ② keep the bias +ve eg 0.01
 ③ don't use ReLU

use alternatives

① Leaky ReLU $\rightarrow$



$$f(x) = x$$

$$f(z) = \max(0.01z, z)$$

$$z \geq 0 \rightarrow z$$

$$z < 0 \rightarrow \frac{1}{10} \times z$$

advantages $\rightarrow$ non saturated, computationally inexpensive, close to 0-centered. ¹⁰⁰ no dying ReLU,

② Parametric ReLU $\rightarrow f(x) = \begin{cases} x & \text{if } x \geq 0 \\ ax & \text{otherwise} \end{cases}$

just same like Leaky ReLU

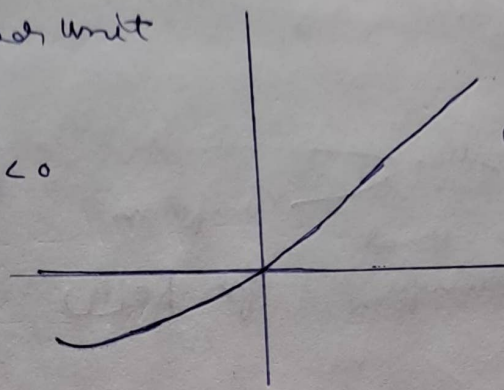
$a \rightarrow$ is a trainable parameter

③ ELU $\rightarrow$ exponential linear unit

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x < 0 \end{cases}$$

$\alpha \rightarrow$ constant
[0.1 $\rightarrow$ 0.3]

better than ReLU



advantages $\rightarrow$ 0-centered
 $\rightarrow$ better generalizes.
disadvantage $\rightarrow$
computationally expensive.

④ Selu $\rightarrow$ Scaled exponential linear unit

$$\text{SELU}(x) = \gamma \begin{cases} x & \text{if } x > 0 \\ \alpha e^x - \alpha & \text{if } x \leq 0 \end{cases}$$

$\alpha \approx 1.67$ $\gamma \rightarrow$ fixed Parameters, not trainable
 $\gamma \approx 1.05$

Weight Initialization Techniques

Wrong weight initialization techniques

① Zero initialization $\rightarrow$ no training as no weight updates in ReLU & Tanh

In case of Sigmoid $\rightarrow$ all the neurons will act like a single neuron; can be proved mathematically

② non-0 - constant value $\rightarrow$ will behave like Sigmoid in the 0 initialization case.

$$a_{11} = a_{12} = a_{13}$$

③ Random initialization $\rightarrow$ ① Small random values

① Tanh / Sigmoid $\rightarrow$ Vanishing gradient problem

② ReLU $\rightarrow$ still workable but very slow convergence

② Large random values $\rightarrow$ slow training in Tanh & Sigmoid Saturated values.

ReLU $\rightarrow$ workable but very big gradient values & hence big jumps in gradient descent

So, what to do?

- ① Xavier/Glorot Initialization $\rightarrow$ normal $\rightarrow$ for tanh / Sigmoid
 $\rightarrow$ uniform
- ② He Initialization $\rightarrow$ normal $\rightarrow$ for ReLU
 $\rightarrow$ uniform

Xavier normal = $\sqrt{\frac{1}{f_{in}}}$ $\rightarrow$ f_{in} = number of inputs coming into a node.

$\swarrow$
 $np.random.randn(n, n) * \sqrt{\frac{1}{f_{in}}}$

He normal = $\sqrt{\frac{2}{f_{in}}}$ $\rightarrow$ same but $* \sqrt{\frac{2}{f_{in}}}$

Uniform ① Xavier = $[-limit, limit]$ $\rightarrow$ range in which the weights will be initialized.
 $limit = \sqrt{\frac{6}{(f_{in} + f_{out})}}$

② He = $[-limit, limit]$
 $limit = \sqrt{\frac{6}{f_{in}}}$

by default it's glort / Xavier - normal.

to set in model. add $(\text{Dense kernel_initializer} = 'he_normal')$

Batch Normalization

An algorithmic method which makes the neural networks train faster & in a stable manner. Activations are normalised for the hidden layers as well.

→ used by / with minibatch GD.

→ used / applied layer-by-layer

for $\rightarrow$ ① $z_{ii} \rightarrow z_{ii}^{\text{Normalized}} \rightarrow g(z_{ii}^N) = a_{ii}$ or
② $z_{ii} \rightarrow g(z_{ii}) \rightarrow a_{ii} \rightarrow a_{ii}^{\text{Normalized}}$

$$z_{ii}^N = \frac{z_{ii} - \mu}{\sigma} \quad \mu = \frac{1}{m} \sum_{i=1}^m z_{ii}^i, \quad m = \text{batch-size}$$
$$\sigma = \sqrt{\frac{1}{m} \sum_{i=1}^m (z_{ii}^i - \mu)^2}$$

$$z_{ii}^i = \frac{z_{ii}^i - \mu}{\sigma + \epsilon}$$

error
term

scale

shift

$$z_{ii}^N = \gamma z_{ii}^N + \beta \rightarrow \gamma \text{ \& \> } \beta \text{ are learnable parameters.}$$

→ occurs individually on every neuron of the chosen layer & occurs for every data point individually but the μ & σ calculated is for the whole batch which is then used by every data point in that batch.

- Advantages $\rightarrow$
- ① more stable training
 - ② faster training
 - ③ acts as a regularizer (mild)
 - ④ reduces the necessity of weight initialization.

model.add(BatchNormalization()) $\rightarrow$ implementation.

Optimizers

Challenges $\rightarrow$ ① Saddle Point ② local minima ③ no learning rate scheduler.

Exponentially Weighted Moving Average $\rightarrow$

It is a technique using which, you can find trends in a time series based data

data day / date wise

used in $\rightarrow$ financial & time series kind of data

we will value the next points more than the current point & older point value decreases over time.

$$V_t = \beta V_{t-1} + (1-\beta) O_t \quad \rightarrow \text{data at time } t$$

EWMA
at time t

$$\beta = \text{constant} \quad 0 < \beta < 1$$

$$V_0 = O_0$$

β determines of how many previous days are you calculating the average ~~of~~

$$\beta \rightarrow \frac{1}{(1-\beta)}$$

$$\therefore \beta = 0.5 \Rightarrow \frac{1}{1-0.5} = \frac{1}{0.5} = 2 \text{ days}$$

The average of 2 days.

$$\text{optimal } \beta = 0.9$$

Momentum optimization $\rightarrow$ if the last x number of gradients are making updates & making you move in a particular direction so, you'll increase your speed of minimization & move faster in that direction.
Like a ball gaining more momentum while going down hill.

Weight update formula $= w_{t+1} = w_t - v_t$

$v_t = \text{velocity}$.

$$v_t = \beta * v_{t-1} + \eta \nabla w_t$$

$0 < \beta < 1$ $\rightarrow$ decay factor

If $\beta = 0$, momentum becomes SGD

& its types.

Issues with momentum $\rightarrow$ ① faster than GD but slower than the others

Nesterov Accelerated Gradient (NAG) Optimizer $\rightarrow$ An improvement over momentum. you reduce the oscillations.

Disadvantage $\rightarrow$ Can get stuck in a local minima.

Implementation = tf.keras.optimizers.SGD $\rightarrow$ learning_rate = , momentum = 0.0, nesterov = True / False, name = "SGD"
 $\rightarrow$ decay batch

Ada Grad $\rightarrow$ adaptive gradient $\rightarrow$ variable / adaptive learning rate. works good on non-scaled data or on sparse features (most inputs 0)

$$w_{t+1} = w_t - \frac{\eta \nabla w_t}{\sqrt{v_t + \epsilon}}$$

$\downarrow$
small no. to prevent 0

$$v_t = v_{t-1} + (\nabla w_t)^2$$

Small for big gradient, big for small gradient $\leftarrow$ for individual / different features

Dis Disadvantage $\rightarrow$ we can reach near the minima but never the minima as learning rate decreases

RMSProp $\rightarrow$ Root Mean Square Propagation

Improvement over AdaGrad

$$V_t = \beta V_{t-1} + (1-\beta)(\nabla w_t)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{V_t + \epsilon}} \nabla w_t \quad \beta = 0.95 \text{ roughly; can be some other}$$

exponentially decaying average

Disadvantage $\rightarrow$ No disadvantages

Adam Optimizer $\rightarrow$ Adaptive moment estimation

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{V_t + \epsilon}} * m_t$$

where,

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) \nabla w_t \rightarrow \text{momentum}$$

$$V_t = \beta_2 V_{t-1} + (1-\beta_2)(\nabla w_t)^2$$

after calculating this, calculate bias correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^{\oplus}}, \quad \hat{V}_t = \frac{V_t}{1 - \beta_2^{\oplus}}$$

$$\beta_1 = 0.9$$

$$\beta_2 = 0.99$$

$\oplus$ = epoch number.

Hyperparameter Tuning

Keras Tuner
on GitHub

Convolutional Neural Network [CNN]

Also called Convnet, used to process data that has a known grid-like-topology like time series or images

In ANN, we use matrix multiplication.

In CNN, we use Convolution

- Convolution layer
- Pooling layer
- fully connected (FC) layer (Dense layer)

Why not use ANN?

- high computation cost
- overfitting
- loss of important information like spatial arrangement of pixels.

Convolution layers → basically filters which extract ~~mathematical filters~~ ^{features / patterns in your} image.

primitive features → edges

Applications → ① Image Classification

② Object localization → Rectangular box around object

③ Object detection

⑤ Image Segmentation

④ Facial Recognition

⑥ Super Resolution.

edges → change in intensity

Convolution Operation

* → convolve operation

filter on top of image & elementwise multiplication then addition

$$-1 \times 0 + -1 \times 0 + -1 \times 0 + +1 \times 0 = 0$$

0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
255	255	255	255	255	255
255	255	255	255	255	255
255	255	255	255	255	255

*

-1	-1	-1
0	0	0
1	1	1

=

0	0	0	0
255	255	255	255
255	255	255	255
0	0	0	0

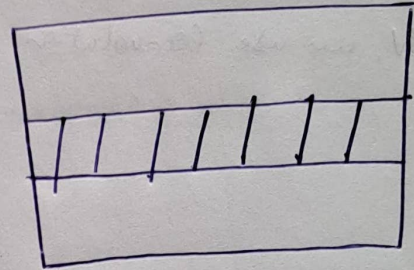
feature map

6x6 image

this one is a horizontal edge detector. usually 3x3 matrix of detector numbers

-1	0	1
-1	0	1
-1	0	1

Vertical edge detector =



The filter values in a CNN are the same as weights in ANN. We initialize the values randomly & the backpropagation sets the values optimal.

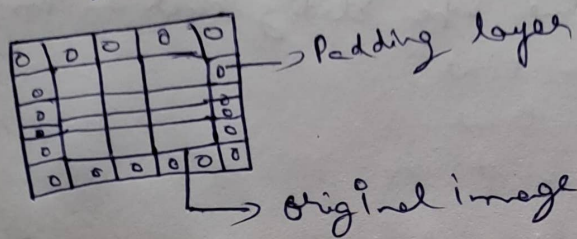
Size of feature map = $\begin{matrix} \text{image} \\ (n \times n) \end{matrix} \begin{matrix} \text{filter} \\ (m \times m) \end{matrix} \rightarrow \begin{matrix} \text{feature map} \\ (n-m+1) \times (n-m+1) \end{matrix}$

For an RGB image of $6 \times 6 \times 3$, the filter will be of $3 \times 3 \times 3$, the feature map is of a single channel, 4×4 for $6 \times 6 \times 3$ each filter will have a feature map

Padding

- ① The feature map is smaller in size as compared to the image. Loss of information.
- ② The edge/side pixels become part of less convolves as compared to the center pixels.

Solution $\rightarrow$ padding $\rightarrow$ adding rows / columns in the original image



generally the pixel values of the padding layer is 0.

new formula for the size of the filter map

$$(n + 2P - f + 1)$$

n = image size, f = filter size
 P = padding size.

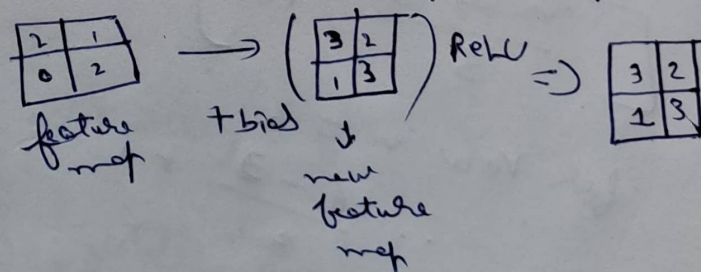
In Keras $\rightarrow$ padding = "valid" $\rightarrow$ means no padding

padding = "same" $\rightarrow$ means automatically figure padding size.

Why do we need strides $\rightarrow$ ① When we just want the high level features of an image.
 ② Saving computing power.

After convolution & before passing the feature map through an element wise ReLU function we add a bias in each element of the feature map. eg if bias = 1 the

every filter has its own bias



Note $\rightarrow$ after the feature map is calculated, we run a ReLU on it so the values > 0 become the only non 0 values & the values < 0 become 0.

Strides

How far the filter moves each time when sliding over an image

Stride = 1 $\rightarrow$ move one step at a time

Stride = 2 $\rightarrow$ move 2 steps at a time

Stride = (1, 1) $\rightarrow$ one right, one down

higher the stride $\rightarrow$ smaller the feature map

$$\left\lfloor \frac{n + 2P - f}{S} + 1 \right\rfloor$$

$S = \text{Stride}$

Strided convolution $\rightarrow$ Stride $\neq 1$

Special case $\rightarrow$ image = 7×6
Stride = 2 filter = 3×3

then, feature map = 2×3

as, not enough pixels on the bottom to place the filter

$\rightarrow$ image = 6×7 , Stride = 2, filter = 3×3

$$\left\lfloor \frac{n - f}{S} + 1 \right\rfloor \rightarrow \text{without padding}$$

for row $n = 6$

$$\therefore \frac{6 - 3}{2} + 1 = \textcircled{1.5} + 1$$

the floor value is taken whenever decimal.

$$= 1 + 1 = 2$$

$$\text{for column } = 3 = \left\lfloor \frac{7 - 3}{2} + 1 \right\rfloor = \underline{2} + 1 = 3$$

$$\therefore 2 \times 3$$

Pooling Layer in CNN

Problem with convolution ① Memory issue ② Translation Variance

Translation Variance $\rightarrow$ moving the input to a different position, changes its output
 $\downarrow$
features location dependent

$\uparrow$ reducing the feature map size
Pooling $\rightarrow$ down sampling your feature map, the features become location independent, the system becomes translation invariant

Parameters of Pooling $\rightarrow$ ① Size (2, 2)
② stride = (2, 2)
③ Type = max pooling

If feature map is this with above defined pooling $\rightarrow$ Taking only the most dominant features in a field.

3	5	1	3
1	2	0	2
1	4	2	3
4	7	5	4

4x4
map

$\rightarrow$

5	3
7	4

2x2
max Pooling

$\rightarrow$ Maximum number of each 2x2 section.

feature size reduced.

Advantages $\rightarrow$ ① Reduces feature map size
② Translation invariance
③ enhanced features (in max pooling)
④ No training needed

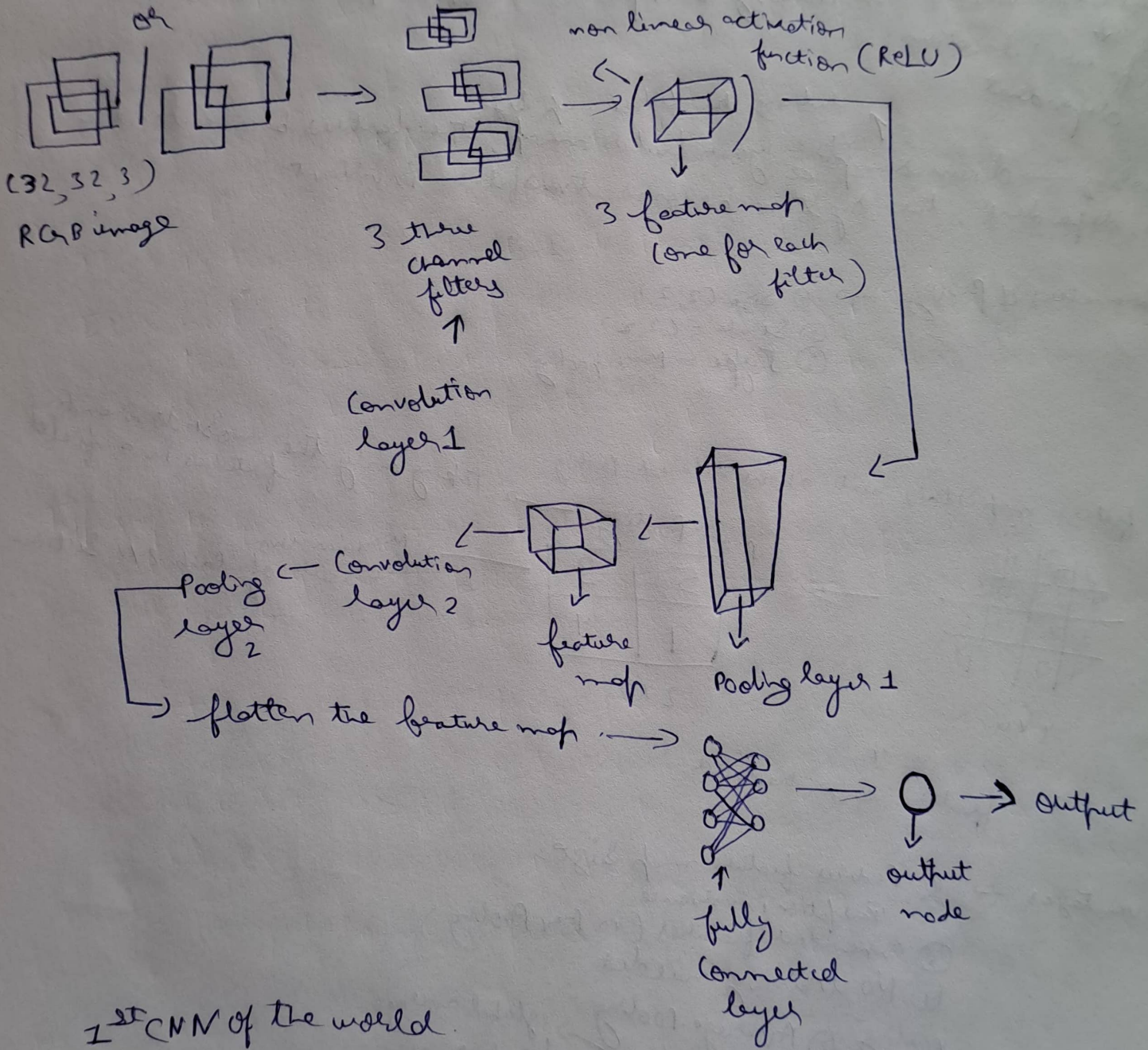
Types $\rightarrow$ ① max pooling ③ Average pooling $\rightarrow$ global average.
② min pooling ④ global pooling $\rightarrow$ global max

Disadvantages $\rightarrow$ ① not used in image segmentation
② information loss

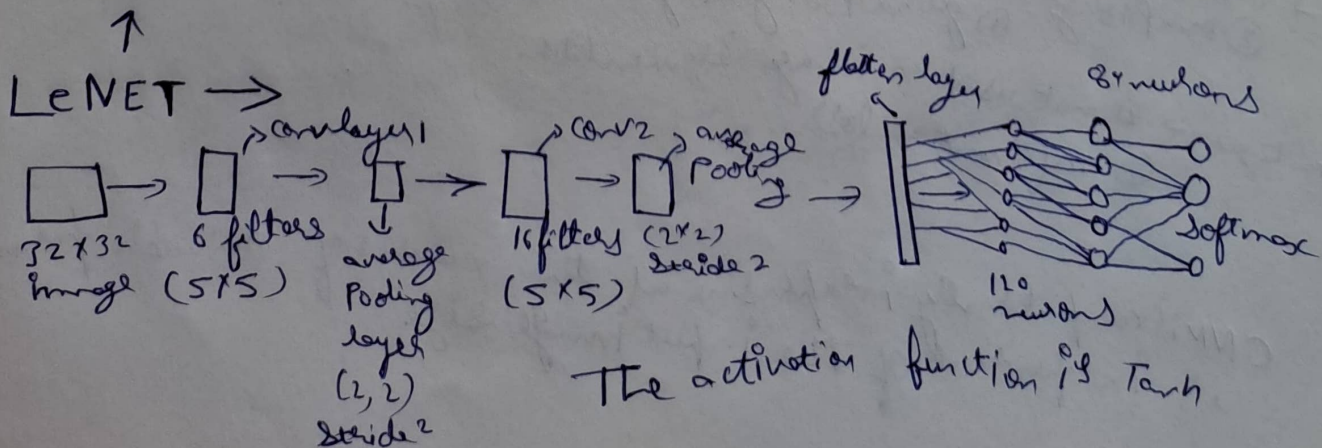
CNN is computationally inexpensive as the number of trainable parameters is independent of the input image size.

Basic CNN Architecture

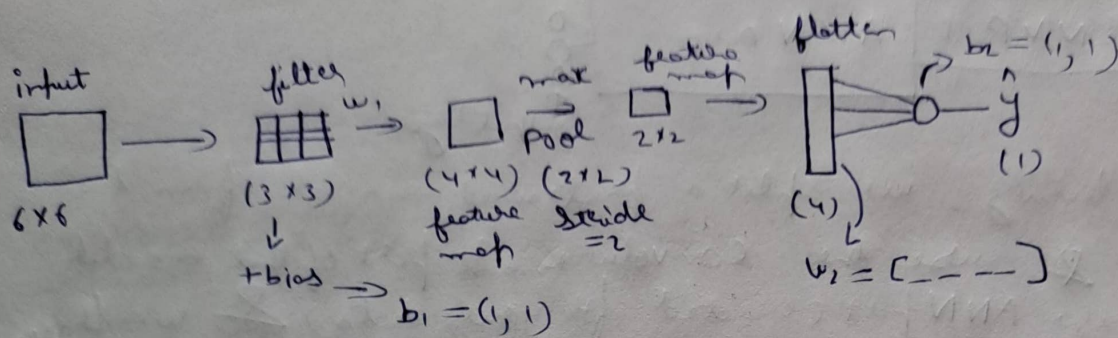
input



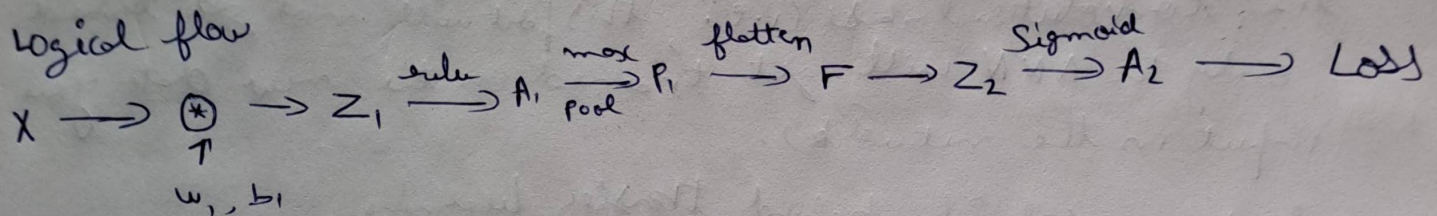
1st CNN of the world



Back Propagation in CNN



Logical flow



forward Propagation

$$Z_1 = \text{Conv}(X, W_1) + b$$

$$F = \text{flatten}(P_1)$$

$$A_L = \hat{y}$$

$$A_1 = \text{ReLU}(Z_1)$$

$$Z_2 = w_2 F + b_2$$

$$P_1 = \text{MaxPool}(A_1)$$

$$A_2 = \sigma(Z_2)$$

Trainable Parameters = $w_1 = (3, 3)$, $w_2 = (1, 4) \Rightarrow 15$ trainable
 $b_1 = b_2 = (1, 1)$

The number of filters in the convolutional layers keeps increasing layer by layer.

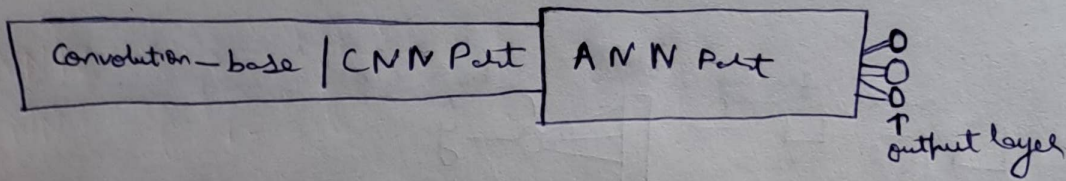
Data Augmentation

Creating more data samples from a single data sample in case of less data availability.

Image Data Generator (parameters)

Transfer Learning

Storing knowledge gained while solving one problem & applying it to a different but related problem.



In transfer learning, we split a pretrained model from the ANN Part & use the Convolution-base part of that model with a new ANN part (created for your data) & we freeze the Convolutional-base (i.e., this part will not get trained, only the ANN will get trained on the new data you input in the network).

This is the whole idea behind transfer learning.

It's better than using pretrained model as it lets you classify even the classes that are non-existent in the pretrained models while being computationally inexpensive.

Types of transfer learning → ① Feature extraction → Explained above, works well when your labels are somewhat similar to the labels the model has ~~at~~ already been trained on.

② Fine tuning → you freeze some convolution layers & let some later convolutional layers as well as your ANN, trained on your data set. Useful when, the pretrained model ^{get} doesn't have any labels even close to the labels you wish to predict. More computationally expensive than feature extraction.

Recurrent Neural Networks [RNN]

ANN $\rightarrow$ for tabular data

CNN $\rightarrow$ for images

RNN $\rightarrow$ for text data

the sequence of the data input matters. eg $\rightarrow$ text / sentences.

RNN is a type of sequential model used to work on sequential data

Why RNNs? $\rightarrow$

- ① Variable input size
- ② Sequence contains some meaning / information
- ③ have memory, so store information of the past

RNN Architecture

Data in RNNs is in the form (time steps, input features)

eg $\rightarrow$

Review	Sentiment
movie was good	1
movie was bad	0
movie was bad not good	0

Vocabulary = 5 words

One hot Encoding

movie	was	good	bad	not
$[10000]$	$[01000]$	$[00100]$	$[00010]$	$[00001]$

Review 1

$[[10000], [01000], [00100]]$

Review 2

$(3, 5)$

Review 3

$(4, 5)$

$(3, 5)$

$\rightarrow$ number of input features

number of time steps

when you send your 1st word time step = 1, for 2nd word, time step = 2 & so on.

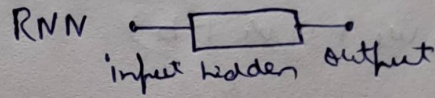
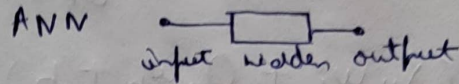
In keras $\rightarrow$ Simple RNN $\rightarrow$ (batch-size, time steps, input-features)
 $(3, 4, 5)$ 4 as it's the

Architecture →

Difference b/w ANN & RNN

Review	Sentiment
--------	-----------

x_1 movie good	1
x_{11} movie was bad	0
x_{12} movie was not good	0
x_{21} movie was good	1
x_{22} movie was bad	0
x_{23} movie was not good	0
x_{31} movie was good	1
x_{32} movie was bad	0
x_{33} movie was not good	0
x_{34} movie was good	1



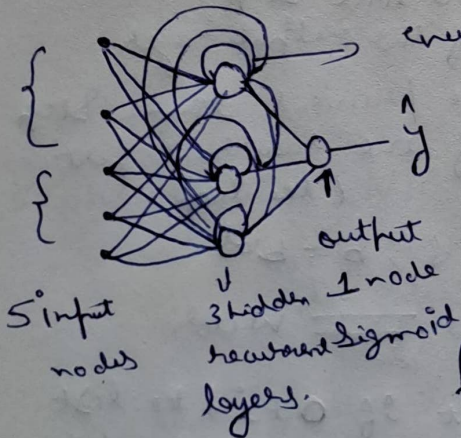
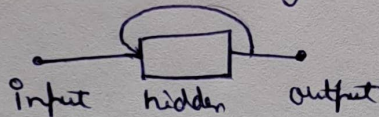
In RNN, you input 1 by 1 on the basis of time.

x_{11} at $t=1$ → output
 x_{12} at $t=2$ → output
 x_{13} at $t=3$ → output

ANNs are a feed forward networks.

RNNs have a concept of state.

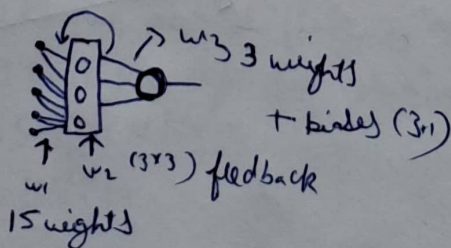
The hidden layer sends a feedback



every node's output will work as an input as well for it & the other nodes during the next input

suppose you got the output for x_{11} , now, that output will go as an input in the RNN along with x_{12} for x_{12} 's output.

for x_{11} we send a random input along with x_{11} . could be 0 or some random numbers.



27 weights + 4 biases
 Trainable parameters.

This hidden RNN layer also has an activation function, by default tanh.

forward Propagation

at $t=1$

$$O_1 = \tanh(X_{11}w_i + O_0w_h + b)$$

at $t=2$

$$O_2 = \tanh(X_{12}w_i + O_1w_h + b)$$

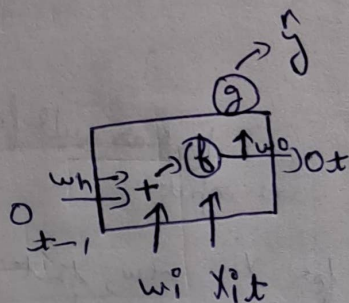
at $t=3$

$$O_3 = \tanh(X_{13}w_i + O_2w_h + b)$$

now, $O_3w_2 + b = Z$

Sigmoid(Z) = prediction

Simplified Representation



i = row number; t = time step

O_t = output of the current time step

w_o = weight matrix of the output layer

O_{t-1} = output of the previous time step

w_h = weight matrix of the recurrent part

w_i = weight matrix

f, g = activation functions; b = bias

$$f(X_{it}w_i + O_{t-1}w_h + b) \rightarrow \text{hidden vector}$$

$$\hat{y} = g(O_t w_o + b)$$

O_t if O_{t-1} is the last output of the hidden layer then

Types of RNN →

- ① Many to one → input → sequence output → non sequential (scalars)
eg → sentiment analysis, rating prediction
- ② One to many → input → non sequential output → sequential (words / sentences / time series)
eg → image captioning, music generation
- ③ Many to many → input outputs both sequential.
Seq 2 Seq eg → parts of speech tagging

Types → ① Same length many to many → eg → parts of speech tagging
② Variable length many to many
↳ translator

- ④ One to one → input output both scalars. eg → image classification
Technically, it's not an RNN rather an ANN or single
step input/output, no feedback ~~or CNN~~

Problems with RNN → ① Problem of long term dependency →
if the number of time steps is very high, the RNN will forget the
older values due to vanishing gradients.

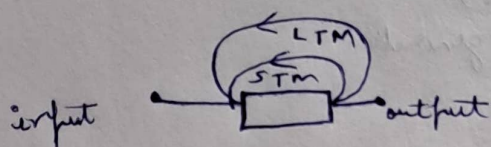
② Stagnated training → due to exploding gradient problems.

Solutions ① → ① Trying out different activation functions
② Better weight initialization techniques.
③ Skip RNNs
④ LSTM

② ① gradient clipping
② controlled learning rate
③ LSTM

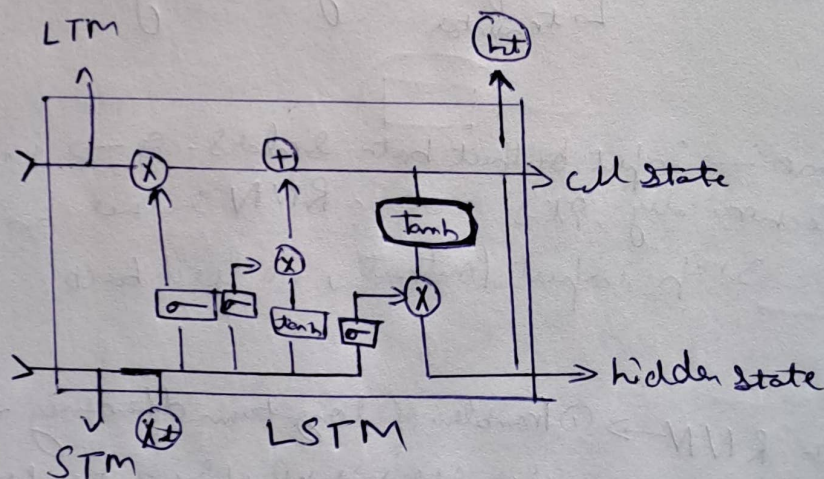
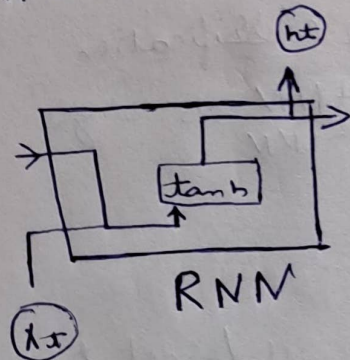
Long Short Term Memory [LSTM]

Core idea $\rightarrow$ In LSTM, we maintain 2 paths, a short term path, a long term path & going forward if we think that any particular information is important, we put it on the long term chain & it stays ~~there~~ there unless you remove it & if it's not removed, it reaches the end of the network.



Difference b/w RNN & LSTM $\rightarrow$ ① 2 memories
② difficult architecture to facilitate the communication b/w both the memories.

Architecture $\rightarrow$



Long term memory (LTM) is also called cell state
Short term memory (~~STM~~) is also called hidden state

In LSTM at $t = t_1$

input = ① previous cell state ② previous hidden state ③ input for the current time step. (for t_1)

output = ① current hidden state ② current cell state.

Processing = ① cell state updation
② h_t (current state output) calculation

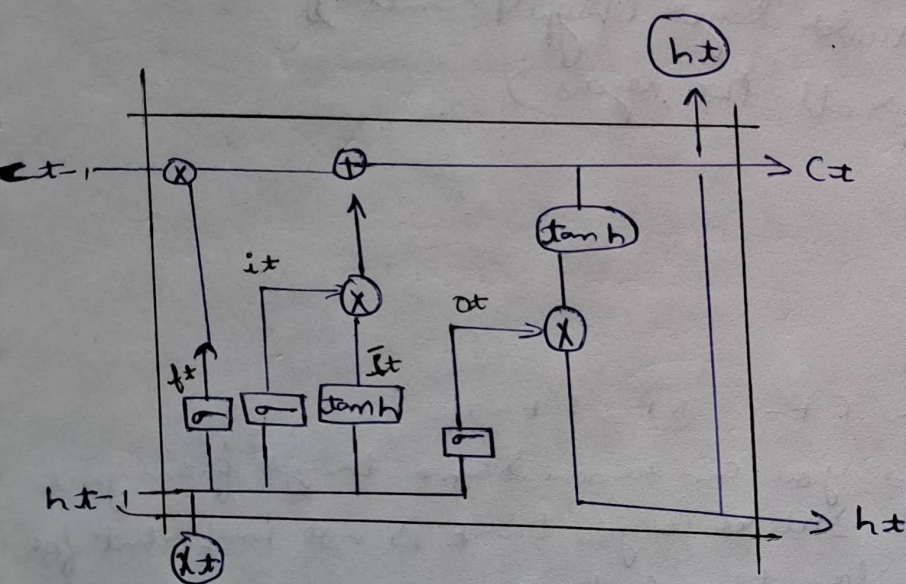
In cell state updation $\rightarrow$ ① what to delete from LTM based on current input
② what to add in LTM based on current input.

Gates $\rightarrow$ The LSTM architecture is divided into 3 gates

- ① forget gate — to remove something from the cell state
- ② input gate — to add something to the cell state

} This combined is the cell state updation

- ③ output gate — to calculate the hidden state h_t



h_t & c_t are vectors of equal dimension

x_t is also a vector

$\rightarrow$ candidate cell state

f_t
in forget gate

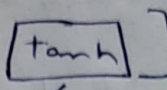
i_t $\bar{c}_t$
in input gate

o_t
in output gate

all of them are vectors, their dimension is the same as that of h_t & c_t

Pointwise operations ($\otimes$, $\oplus$, $\tanh$) occur b/w 2 vectors

- $\otimes \rightarrow$ element wise multiplication
- $\oplus \rightarrow$ element wise addition
- $\tanh \rightarrow$ element wise tanh

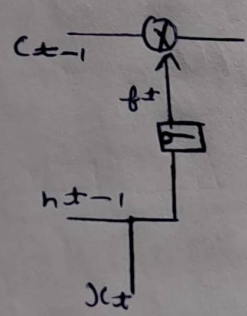
Neural network layers [ , ]

neural network layer containing many nodes with a activation function

Same with tanh activation layer

[You choose the number of nodes in a particular layer (hyperparameter) (Same for all the layers)]

Forget Gate $\rightarrow$



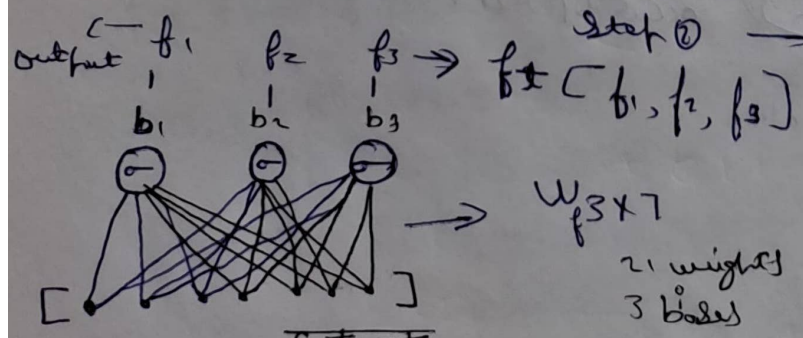
input $\rightarrow c_{t-1}, x_t, h_{t-1}$

output $\rightarrow$ you can remove some things from your all state which you think is not important for the long term context

for eg $\rightarrow$
 number of neural nodes = 3
 $x_t = 4D$ vector $[x_{i1}, x_{i2}, x_{i3}, x_{i4}]$

$(c_t, h_t) = 3D$ vectors
 $\hookrightarrow$ as their dimension = no. of nodes in the neural layer

2 step process $\rightarrow$ Step ① $\rightarrow$ f_t calculation



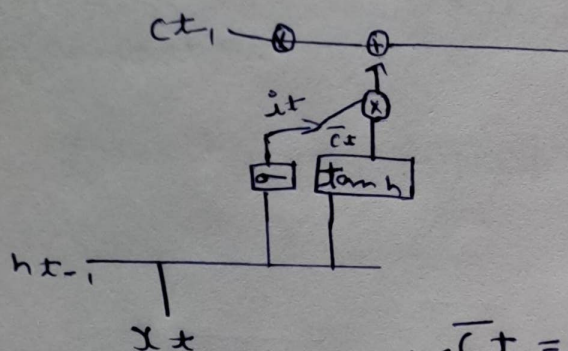
Step ② $\rightarrow c_{t-1} \otimes f_t$

this is the removal from the cell state

$[h_{t-1}, x_t] = [7 \times 1]$
 Concatenation

$$f_t = \sigma(w_f [h_{t-1}; x_t] + b_f)$$

② Input gate -



works in 3 stages

- ① calculate candidate cell state $\bar{c}_t$
- ② calculate i_t
↳ decides what from $\bar{c}_t$ should actually be added to LTM
- ③ calculate $c_t \rightarrow$ current cell state

things that should be added to LTM

$$\bar{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$$

Potential important information

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

actual important information to be stored in LTM

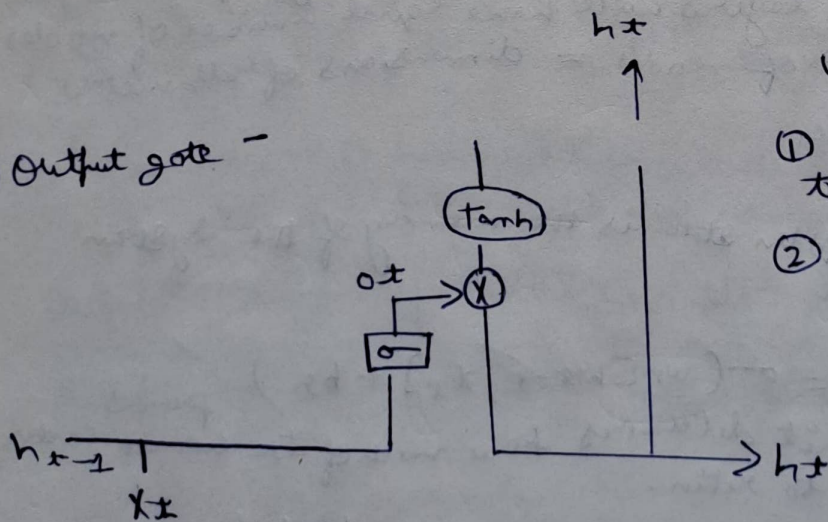
$i_t \otimes \bar{c}_t \rightarrow$ filtered ~~candidate cell~~ candidate cell state

$$c_t = f_t \otimes c_{t-1} \oplus i_t \otimes \bar{c}_t$$

↓
LTM

solves vanishing gradient problem

③ Output gate -



works in 2 steps

- ① elementwise $\tanh$ on c_t to bring the values between -1 & 1
- ② $h_t = [o_t \otimes \tanh(c_t)]$

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \otimes \tanh(c_t)$$

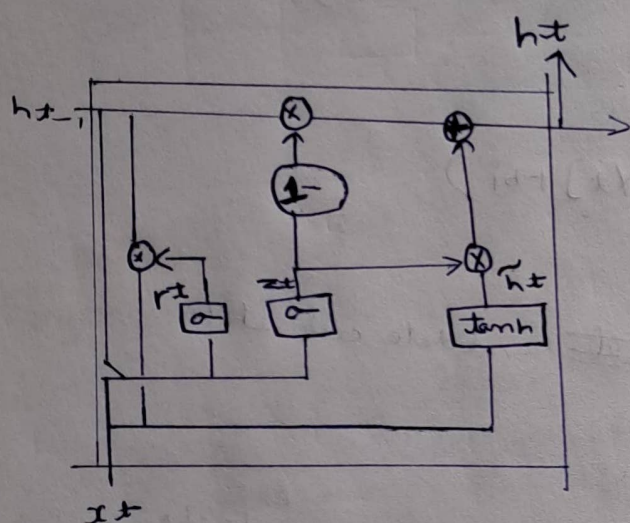
Output for the current timestep.

Grated Recurrent Unit [GRU]

An RNN architecture used to process sequential data

Simpler in architecture as compared to LSTM, hence faster training, results comparable to LSTM

Architecture $\rightarrow$



h_{t-1} = previous hidden state

h_t = current hidden state

x_t = current input

r_t = reset gate

z_t = update gate

$\tilde{h}_t$ = candidate hidden state

all of these are vectors. The dimensions of all of them are the same, except for x_t .

σ , $\tanh$ $\rightarrow$ neural network (fully connected) layers.
 $\rightarrow$ number of hidden units/nodes is a hyperparameter
 $\rightarrow$ all of the layers will have equal number of nodes except x_t
 $\rightarrow$ the number of nodes = dimensions of all vectors

$\otimes \rightarrow$ Pointwise multiplication

$\oplus \rightarrow$ Pointwise addition

$\mathbb{1} \rightarrow$ pointwise 1-

hidden state is the memory of the system

step ① Calculate r_t

step ② Calculate $\tilde{h}_t$

step ③ Calculate z_t

step ④ Calculate h_t

$$r_t = \sigma(w_r[h_{t-1}, x_t] + b_r)$$

$\rightarrow$ it determines how much of the ^{previous} context to retain.

$r_t \otimes h_{t-1} \rightarrow$ modulated / reset memory

$$\tilde{h}_t = \tanh(w_c[r_t \otimes h_{t-1}, x_t] + b_c)$$

$$z_t = \sigma(w_z[h_{t-1}, x_t] + b_z)$$

how to balance b/w h_{t-1} & h_t , which to value more

$$h_t = (1 - z_t) \oplus h_{t-1} \oplus z_t \otimes \tilde{h}_t$$

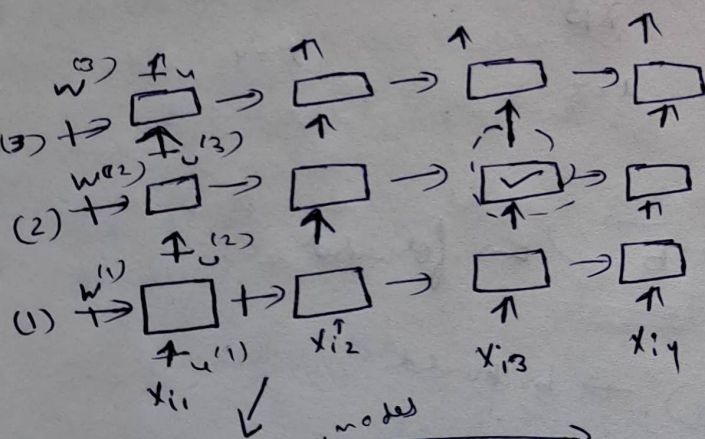
LSTM VS GRU

- ① LSTM has 3 gates, GRU has 2 gates
- ② LSTM has 2 states ① hidden ② cell, GRU has only 1 hidden state
- ③ LSTM has more parameters, GRU has less number of parameters
- ④ LSTM is architecturally more complex, GRU is less complex
- ⑤ Comparable performance

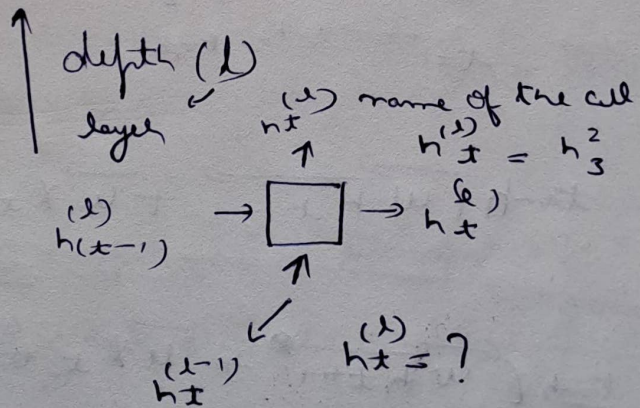
Deep RNNs / Stacked RNNs

Architecture →

number of neurons/units in each layer can be different



eg ① 1st layer = 2 nodes
 ② 2nd layer = 3 nodes
 ③ 3rd layer = 4 nodes



$$h_t^{(l)} = \tanh(w^{(l)} h_{t-1}^{(l)} + u^{(l)} x_t^{(l)} + b)$$

use case → ① hierarchical representation
 ② Customization for advanced tasks

Deep LSTMs & Deep GRUs can also be implemented.

Disadvantages → ① possible overfitting ② increased training time.

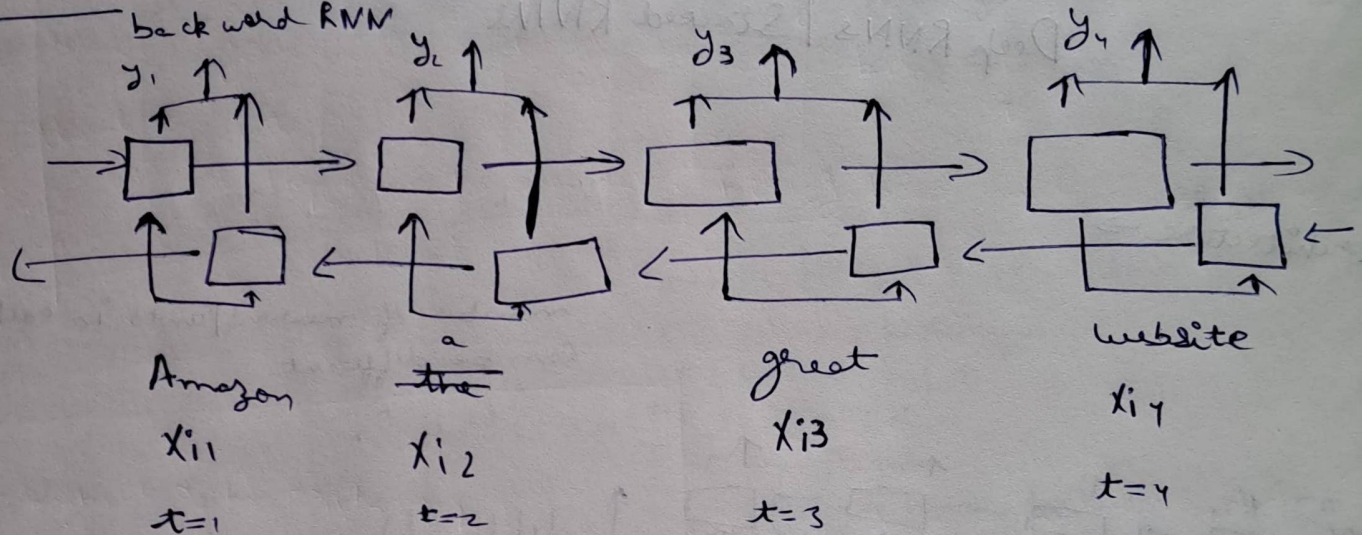
Bidirectional RNNs

Amazon a great website

Amazon a great river

→ forward RNN

← backward RNN



$$\vec{h}_t = \tanh(\vec{w} \vec{h}_{t-1} + \vec{w} x_t + b) \rightarrow \text{forward}$$

$$\overleftarrow{h}_t = \tanh(\overleftarrow{w} \overleftarrow{h}_{t+1} + \overleftarrow{w} x_t + b) \rightarrow \text{backward}$$

$$y_t = \sigma(W[\vec{h}_t, \overleftarrow{h}_t] + b)$$

Bidirectional LSTM & GRUs also exist

Applications →

- ① Named entity recognition
- ② Parts of Speech tagging
- ③ Machine translation
- ④ Time Series forecasting

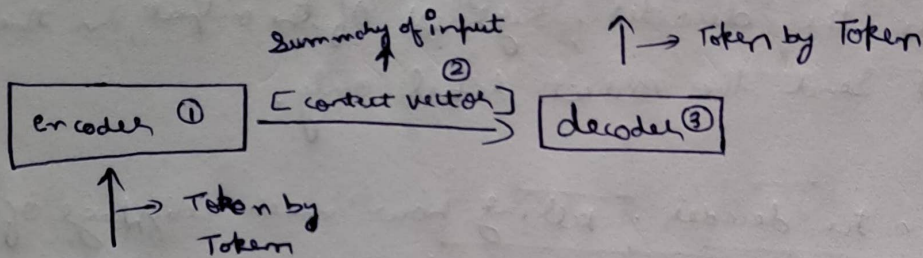
Drawbacks →

- ① Complexity
- ② You might not have complete data in one go,

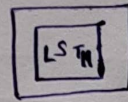
Encoder - Decoder / Sequence - to - Sequence Architecture

High level overview $\rightarrow$

आप से मिल कर अच्छा लगा



Nice to meet you $\rightarrow$ input sequence



We use an LSTM cell inside an encoder
GRU can be used as well.

We use another LSTM cell inside a decoder

The final h_t , c_t of the encoder become the initial h_t , c_t of the decoder.

In decoder, the $x-1$ time steps output goes as an input at x timestep

$\langle \text{Start} \rangle \rightarrow$ Special symbol telling the decoder to start producing the output.

$\langle \text{End} \rangle \rightarrow$ Special symbol telling the decoder to stop producing the output.

decoder has a Softmax Layer as well.